

Conditionally Trivial Special Member Functions

Document #: P0848R2
Date: 2019-06-16
Project: Programming Language C++
CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Casey Carter
<casey@carter.net>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Proposal](#)
- [4 Wording](#)
- [5 Acknowledgments](#)
- [6 References](#)

§ 1 Revision History

[[P0848R0](#)] was presented to EWG at the San Diego meeting (November 2018). It proposed to change the rules for trivially copyable such that we only consider the *best viable candidate* amongst the copy constructors given a synthesized overload resolution. That is, depending on `c` `<T` `>`, we either consider only `#2` (because `#1` wouldn't be viable) or only `#1` (because it would be more constrained than `#2` and hence a better match). However, EWG considered this to be confusing as trivially copyable is a property of a type and adding overload resolution simply adds more questions about context (e.g. do we consider accessibility?). EWG requested a new mechanism to solve this problem.

[[P0848R1](#)] was also presented to EWG in San Diego, proposed to introduce an intermediate layer: a constructor for a class `X` of the form `X` `(X` `const` `&)` rather than being considered a copy constructor would instead become a *prospective copy constructor*, and a *copy constructor* would be any prospective copy constructor whose constraints are satisfied and is at least as constrained as every other prospective copy constructor. In this way, we don't change the definition of trivially copyable at all - we change the definition for each of the special member functions.

During Core wording review in the Kona meeting (February 2019), a new direction was suggested that is somewhat of a compromise between the two other positions: we use the “constraints are satisfied and is at least as constrained as” rule on the special member functions, but we only apply it to the trivially copyable rules. This ensures that we maintain these as properties of the type rather than properties of an expression - and is the direction this paper takes.

§ 2 Introduction

For a complete motivation for this proposal, see [P0848R0]. In brief, it is important for certain class template instantiations to propagate the triviality from their template parameters - that is, we want wrapper `<T>` to be trivially copyable if and only if `T` is copyable. In C++17, this is possible, yet is extremely verbose. The introduction of Concepts provides a path to make this propagation substantially easier to write, but the current definition of trivially copyable doesn't quite suffice for what we want.

Consider:

```
template <typename T>
concept C = /* ... */;

template <typename T>
struct X {
    // #1
    X(X const&) requires C<T> = default;

    // #2
    X(X const& ) { /* ... */ }
};
```

According to the current working draft, both `#1` and `#2` are copy constructors. The current definition for trivially copyable requires that *each* copy constructor be either deleted or trivial. That is, we always consider both copy constructors, regardless of `T` and `C<T>`, and hence no instantiation of `X` is ever trivially copyable.

This paper suggests that those specializations `X<T>` for which `T` satisfies `C` should be considered trivially copyable - this very logically follows the model of constraints and would make it substantially easier to write class templates that are conditionally trivially copyable. Indeed, it would become actually easy to do such a thing rather than quite complex with multiple layers of conditional base classes.

§ 3 Proposal

The current relevant definitions in [class.prop] read:

A trivially copyable class is a class:

- where each copy constructor, move constructor, copy assignment operator, and move assignment operator ([class.copy.ctor], [class.copy.assign]) is either deleted or trivial,
- that has at least one non-deleted copy constructor, move constructor, copy assignment operator, or move assignment operator, and
- that has a trivial, non-deleted destructor.

A *trivial class* is a class that is trivially copyable and has one or more default constructors ([class.default.ctor]), all of which are either trivial or deleted and at least one of which is not deleted.

There are two aspects of these definitions that need to be changed for this proposal: one that applies to five of the special member functions in the same way (the copy and move constructors and assignment operators and the default constructor), and one that applies specifically to the destructor.

For the five non-destructor special member functions, we introduce a new concept called an *eligible special member function* - which is a special member function that is:

- not deleted
- has all of its constraints (if any) satisfied
- no special member function of the same kind, with the same first parameter type (except for the default constructor), is more constrained

And then simplify the definitions of *trivially copyable* and *trivial* to use eligible special members instead of just any special members.

For the destructor, we go in a slightly different direction. We introduce a sub-classification called a *prospective destructor* which is simply any function declared for a class *x* that is spelled

`~X()` with some constraints, and then redefine destructor to be the eligible destructor, requiring that there be only one.

Using the example from R0:

```
template <typename T>
struct optional {
    // #1
    optional(optional const&)
        requires TriviallyCopyConstructible<T> && CopyConstructible<T>
        = default;

    // #2
    optional(optional const& rhs)
        requires CopyConstructible<T>
        : engaged(rhs.engaged)
    {
        if (engaged) {
            new (value) T(rhs.value);
        }
    }
};
```

For all specializations, we have two copy constructors:

`T` `=unique_ptr` `<` `int`
 satisfied so there is no eligible copy constructor. For `T`
`#2` has its constraints satisfied, so only
`T` `=` `int`, both `#1` and
`#1` is more constrained than `#2`, so only

`#1` and `#2`. For
`>`, neither copy constructor has its constraints
`=std` `::string`, only
`#2` is an eligible copy constructor. For
`#2` have their constraints satisfied.
`#1` is an eligible copy constructor.

§ 4 Wording

Change 6.6.7 [class.temporary] to refer to the future definition of eligibility:

- 3 When an object of class type `x` is passed to or returned from a function, if each **eligible** copy constructor ([class.prop]) ~~;~~ and move constructor is trivial, the destructor of `X` is either trivial or deleted, and `x` has at least one ~~non-deleted~~ **eligible** copy or move constructor, implementations are permitted to create a temporary object to hold the function parameter or result object. The temporary object is constructed from the function argument or return value, respectively, and the function's parameter or return object is initialized as if by using the ~~non-deleted~~ **eligible** trivial constructor to copy the temporary (even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object).

And likewise for 7.6.1.2 [expr.call]:

- 12 [...] Passing a potentially-evaluated argument of class type having a non-trivial **eligible** copy constructor ([class.prop]), a non-trivial **eligible** move constructor, or a non-trivial destructor, with no corresponding parameter, is conditionally-supported with implementation-defined semantics.

Change 9.4.2 [dcl.fct.def.default] to account for prospective destructors that aren't destructors:

- 5 Explicitly-defaulted functions and implicitly-declared functions are collectively called *defaulted* functions, and the implementation shall provide implicit definitions for them ([class.ctor], [class.dtor], [class.copy.ctor], [class.copy.assign]), which might mean defining them as deleted. **A defaulted prospective destructor ([class.dtor]) that is not a destructor shall be defined as deleted. A defaulted special member function that is not an eligible special member function ([class.prop]) shall be defined as deleted.** A function is *user-provided* if it is user-declared and not explicitly defaulted or deleted on its first declaration.

Insert at the beginning of 11.1 [class.prop] a definition of eligibility:

- 0 **An *eligible special member function* is a special member function ([special]):**
 - (0.1) **— that is not deleted,**
 - (0.2) **— where each of its associated constraints ([temp.constr]), if any, are satisfied, and**
 - (0.3) **— no special member function of the same kind with the same first parameter type (if any) is more constrained ([temp.constr.order]).**

Change the definitions of *trivially copyable* and *trivial* in 11.1 [class.prop]:

- 1 **A *trivially copyable class* is a class:**
 - (1.1) **— where each **eligible** copy constructor, move constructor, copy assignment operator, and move assignment operator ([class.copy.ctor], [class.copy.assign]) is ~~either deleted or~~ trivial,**

- (1.2) — that has at least one ~~non-deleted~~ eligible copy constructor, move constructor, copy assignment operator, or move assignment operator, and
 - (1.3) — that has a trivial, non-deleted destructor.
- 2 A *trivial class* is a class that is trivially copyable and has one or more eligible default constructors ([class.default.ctor]), all of which are trivial. ~~either trivial or deleted and at least one of which is not deleted.~~

Change 11.3.3 [special] to refer to prospective destructors, and make everything plural:

- 1 ~~The default~~ Default constructors ([class.default.ctor]), copy constructors, move constructors ([class.copy.ctor]), copy assignment operators, move assignment operators ([class.copy.assign]), and prospective destructors ([class.dtor]) are *special member functions*.

Change the definition of destructor as follows.

Change 11.3.6 [class.dtor]:

- 1 In a declaration of a prospective destructor, the declarator is a function declarator (9.2.3.5) of the form [...] A prospective destructor shall take no arguments (9.2.3.5). Each *decl-specifier* of the *decl-specifier-seq* of a prospective destructor declaration (if any) shall be

`friend,` `inline,` or `virtual.`
- a At the end of the definition of a class, overload resolution is performed among the prospective destructors declared in that class with an empty argument list to select the *destructor* for the class. The program is ill-formed if overload resolution fails. Destructor selection does not constitute a reference to ([dcl.fct.def.delete]) or odr-use of ([basic.def.odr]) the selected destructor, and in particular, the selected destructor may be deleted.

Change 11.3.6 [class.dtor]:

- 4 If a class has no user-declared prospective destructor, a prospective destructor is implicitly declared as defaulted (9.4). An implicitly-declared prospective destructor is an inline public member of its class.

An implicitly-declared prospective destructor for a class X will have the form

`~X()`

Change 11.3.6 [class.dtor]:

- 10 A prospective destructor can be declared `virtual` (11.6.2) or pure `virtual` (11.6.3); if the destructor of a class is `virtual` and any objects of that class or any derived class are created in the program, the destructor shall be defined. If a class has a base class with a virtual destructor, its destructor (whether user- or implicitly-declared) is virtual.

§ 5 Acknowledgments

Thanks to Gaby dos Reis, Daveed Vandevoorde, and Jonathan Wakely for helping bring us to this design. Thanks to Jens Maurer and Richard Smith for the lengthy discussions and wording wizardry.

§ 6 References

[P0848R0] Barry Revzin, Casey Carter. 2017. Conditionally Trivial Special Member Functions.
<https://wg21.link/p0848r0>

[P0848R1] Barry Revzin, Casey Carter. 2019. Conditionally Trivial Special Member Functions.
<https://wg21.link/p0848r1>